

0.8.5. #include <diviser> — a / b

#include <diviser> : a / b demande une routine, sauf par 1, 2, 4, 8, 16... écrits en clair.

1. Ce qu'il faut comprendre

Ce que c'est : le processeur **ne sait pas diviser** non plus. Diviser par 2, 4, 8, 16... écrit en clair, c'est décaler les bits : le compilateur le fait seul. Par **tout autre nombre**, ou par une valeur calculée, il faut une routine.

La ligne à écrire : `#include <diviser>`. Le résultat est **entier** : 100 / 7 vaut 14, le reste se perd.

Ce qu'elle coûte : environ **30 octets**.

Essaie : `a / 4` sans la ligne : ça compile.

On retrouve diviser dans 5 leçons

[0.9](#), [0.9.1](#), [0.9.2](#), [10](#), [24](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **406 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

```
démarrage      60 o ← la console, à l'allumage – la mise en route de la console
main()         27 o ← démarrage – ta fonction
VBlank         8 o ← la console, à chaque image – compte les images, 60 fois par seconde
AttendreImage  28 o ← image() – attend l'image suivante
AttendreVBlank 16 o ← démarrage – attend que l'écran soit libre
AttendreAcces  12 o ← EcrireNombre – attend de pouvoir écrire une case
EcrireNombre   33 o ← nombre() – écrit un nombre en chiffres
Diviser        17 o ← main() – divise – le processeur ne sait pas le faire
CopierTuiles   31 o ← démarrage – copie les lettres et les dessins en mémoire vidéo
EffacerCarte   14 o ← démarrage – vide les deux cartes de l'écran
Tuiles        160 o ← CopierTuiles – 10 lettres sur 43
```

Lettres gravées : 0123456789

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, AdresseCase, EffacerCases, EcrireTexte, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, LireManette, DonneesTransfert, Notes

Fonctions du langage appelées :

```
nombre() × 1    10 o sur place, dans main() + EcrireNombre (33 o)
image() × 1     3 o sur place, dans main() + AttendreImage (28 o)
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// ---- #include <diviser> : a / b ----

#include <diviser> // la routine qui divise
#include <nombre>
```

2. UNE VARIABLE — A

```
uint8_t a = 100;
```

3. UNE VARIABLE — B

```
uint8_t b = 7;
```

4. UNE VARIABLE — QUOTIENT

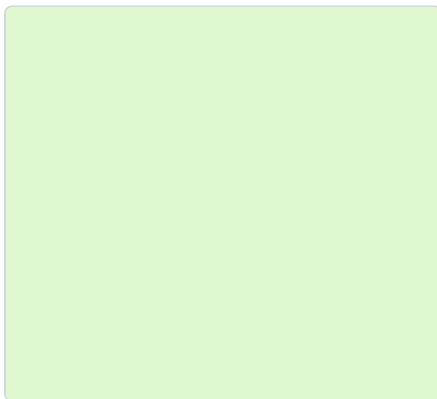
```
uint8_t quotient = 0;
```

5. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

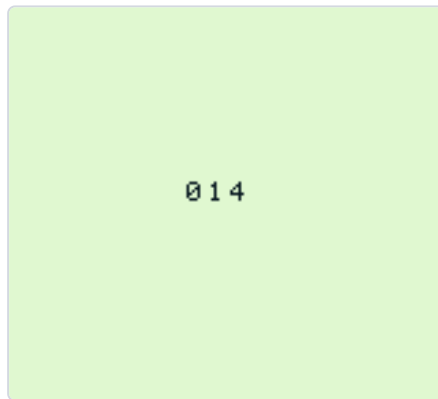
```
int main() {  
    quotient = a / b;           // 100 / 7 = 14 (et il reste 2)  
    nombre(8, 8, quotient);    // 014  
  
    while (true) {  
        image();  
    }  
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

014 au milieu de l'écran.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ 100 / 7 = 14

✓ 014 s'affiche

Ensuite : **0.9. Tout l'alphabet, avec une seule boucle** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++